ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Е.О. Шумова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5.доп

«Виртуальные функции и абстрактные классы»

по курсу: ОСНОВЫ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Условие с вариантом

Вариант №48

№ варианта	Фигура	Расположение
48	5	12

Изучить принципы построения консольных приложений, применив на практике знания базовых синтаксических конструкций языка C++ и объектно-ориентированного программирования.

2. Листинг программы

```
// Файл trapezoid.h
#ifndef TRAPEZOID_H
#define TRAPEZOID_H

#include "shape.h"
#include "line.h"
#include <memory>

// Класс Trapezoid наследуется от класса Shape
class Trapezoid : public Shape
{
private:
    // Уникальные указатели на объекты Line, представляющие стороны трапеции
    std::unique_ptr<Line> _topLeftToTopRight;
    std::unique_ptr<Line> _topRightToBottomRight;
    std::unique_ptr<Line> _bottomRightToBottomLeft;
    std::unique_ptr<Line> _bottomLeftToTopLeft;

public:
    // Конструктор, принимающий координаты вершин трапеции и создающий объекты Line
для сторон
    Trapezoid(const Point& topLeft, const Point& topRight, const Point& bottomLeft,
const Point& bottomRight);

    // Методы для отрисовки и перемещения трапеции
    void draw(Screen* screen) const override;
    void move(Point p) override;

    // Методы для получения координат вершин трапеции
    Point getLeftTop() const;
    Point getRightTop() const;
    Point getLeftBottom() const;
    Point getRightBottom() const;

    // Деструктор по умолчанию
    ~Trapezoid() override;
};

// Реализация конструктора Trapezoid
Trapezoid::Trapezoid(const Point& topLeft, const Point& topRight, const Point&
bottomLeft, const Point& bottomRight)
    : _topLeftToTopRight(std::make_unique<Line>(topLeft, topRight)),
    _topRightToBottomRight(std::make_unique<Line>(topRight, bottomRight)),
```

```
    _bottomRightToBottomLeft(std::make_unique<Line>(bottomRight, bottomLeft)),
    _bottomLeftToTopLeft(std::make_unique<Line>(bottomLeft, topLeft))
{
}

// Реализация метода отрисовки трапеции
void Trapezoid::draw(Screen* screen) const
{
    _topLeftToTopRight->draw(screen);
    _topRightToBottomRight->draw(screen);
    _bottomRightToBottomLeft->draw(screen);
    _bottomLeftToTopLeft->draw(screen);
}

// Реализация метода перемещения трапеции
void Trapezoid::move(Point p)
{
    _topLeftToTopRight->move(p);
    _topRightToBottomRight->move(p);
    _bottomRightToBottomLeft->move(p);
    _bottomLeftToTopLeft->move(p);
}

// Реализация методов получения координат вершин трапеции
Point Trapezoid::getLeftTop() const
{
    return _topLeftToTopRight->getLeftTop();
}

Point Trapezoid::getRightTop() const
{
    return _topRightToBottomRight->getRightTop();
}

Point Trapezoid::getLeftBottom() const
{
    return _bottomLeftToTopLeft->getLeftBottom();
}

Point Trapezoid::getRightBottom() const
{
    return _bottomRightToBottomLeft->getRightBottom();
}

// Реализация деструктора по умолчанию
Trapezoid::~Trapezoid() = default;

#endif // TRAPEZOID_H


// Labor_5D.cpp : Этот файл содержит функцию "main". Здесь начинается и
заканчивается выполнение программы.
//
// Файл main.cpp
#include <iostream>
#include "screen.h"
#include "line.h"
#include "square.h"
#include "trapezoid.h"
#include <vector>
#include <memory>
int main()
{
      system("color F1");
      auto screen = std::make_unique<Screen>(50, 35);
```

```
	std::vector<std::shared_ptr<Shape>> shapes;
	// Уменьшим ширину трапеции
	shapes.emplace_back(std::make_shared<Trapezoid>(Point(10, 0), Point(14, 0),
Point(5, 5), Point(19, 5))); // Трапеция
	Point p1 = shapes[shapes.size() - 1]->getLeftBottom();
	p1.setX(p1.getX() - 2);
	p1.setY(p1.getY() + 1);
	Point p2 = shapes[shapes.size() - 1]->getRightBottom();
	p2.setX(p2.getX() + 2);
	p2.setY(p2.getY() + 1);
	shapes.emplace_back(std::make_shared<Line>(p1, p2)); // Линия под шляпой
	p1 = shapes[shapes.size() - 1]->getLeftBottom();
	p1.setX(p1.getX() + 1);
	p1.setY(p1.getY() + 1);
	p2 = shapes[shapes.size() - 1]->getRightBottom();
	p2.setX(p2.getX() - 1); p2.setY(p2.getY() + 10);
	shapes.emplace_back(std::make_shared<Square>(p1, p2)); // Голова
	Point eyeLeft = shapes[shapes.size() - 1]->getLeftTop();
	eyeLeft.setX(eyeLeft.getX() + 2);
	eyeLeft.setY(eyeLeft.getY() + 2);
	Point eyeRight = Point(eyeLeft.getX() + 1, eyeLeft.getY());
	shapes.emplace_back(std::make_shared<Line>(eyeLeft, eyeRight)); // Левый глаз
	eyeRight = shapes[shapes.size() - 2]->getRightTop();
	eyeRight.setX(eyeRight.getX() - 2);
	eyeRight.setY(eyeRight.getY() + 2);
	eyeLeft = Point(eyeRight.getX() - 1, eyeRight.getY());
	shapes.emplace_back(std::make_shared<Line>(eyeLeft, eyeRight)); // Правый
глаз
	std::shared_ptr<Shape> leftEye = shapes[shapes.size() - 2];
	std::shared_ptr<Shape> rightEye = shapes[shapes.size() - 1];
	Point nose = Point(leftEye->getRightTop().getX() +
		(rightEye->getLeftTop().getX() - leftEye->getRightTop().getX()) / 2,
		leftEye->getRightTop().getY() + 2);
	shapes.emplace_back(std::make_shared<Line>(nose, nose)); // Нос
	std::shared_ptr<Shape> head = shapes[shapes.size() - 4];
	p1 = head->getLeftBottom(); p1.setX(p1.getX() + 2); p1.setY(p1.getY() - 2);
	p2 = head->getRightBottom(); p2.setX(p2.getX() - 2); p2.setY(p2.getY() - 2);
	shapes.emplace_back(std::make_shared<Line>(p1, p2)); // Рот
	auto costume = std::make_shared<Line>(Point(p1.getX(), p1.getY() + 10),
		Point(p2.getX(), p2.getY() + 10));
	shapes.emplace_back(costume); //Линия костюма
	p1 = Point(costume->getLeftBottom().getX(), costume->getLeftBottom().getY() +
1);
	auto leftDot = std::make_shared<Line>(p1, p1);
	shapes.emplace_back(leftDot); //Левая точка
	p1 = Point(costume->getRightBottom().getX(), costume->getRightBottom().getY()
+
		1);
	auto rightDot = std::make_shared<Line>(p1, p1);
	shapes.emplace_back(rightDot); //Правая точка
	for (auto shape : shapes)
		shape->draw(screen.get());
	screen->draw();
	std::cin.get();
	return 0;
}
```

3. Скриншоты с исходными данными и выполнением программы

```
        *****
      *     *
     *       *
    *         *
   *           *
  ***************
******************
 *****************
 *               *
 * **         ** *
 *               *
 *       *       *
 *               *
 *               *
 * ************* *
 *               *
 *****************
 *               *
 *               *
 *               *
```

3. Координаты различных элементов, таких как глаза, нос, рот, точки и линия костюма, вычисляются относительно координат других элементов (например, глаза относительно верхнего левого угла головы).

4. Все созданные фигуры добавляются в вектор shapes, и затем каждая фигура отрисовывается на экране с использованием метода draw.

5. Наконец, вызывается метод screen->draw(), который выводит на экран все добавленные фигуры.

Программа завершается после ввода символа с клавиатуры. Она создает забавный графический рисунок, представляющий персонажа с лицом и костюмом.